

Algoritmos y Programación II

75.41

Cátedra: Lic. Gustavo Carolo

Guía de Estudio – Abstracción

Agosto 2006

Índice

ÍNDICE.....	2
CONCEPTOS GENERALES.....	3
ALGORITMO.....	3
PARADIGMAS DE PROGRAMACIÓN	3
PRINCIPIOS DE PROGRAMACIÓN.....	4
TIPO DE DATOS	4
ESTRUCTURA DE DATOS	4
ABSTRACCIÓN.....	5
TIPO DE DATO ABSTRACTO.....	6
HISTORIA	6
DEFINICIÓN.....	6
CARACTERIZACIÓN	7
<i>Principales características</i>	<i>8</i>
Ocultamiento de la información.....	8
Protección de la manipulación	8
Simplicidad.....	8
Integridad.....	8
Implementación Independiente.....	8
<i>Objetivos.....</i>	<i>8</i>
<i>Ventajas</i>	<i>8</i>
<i>Etapas en la construcción de un TDA.....</i>	<i>8</i>
<i>Representación</i>	<i>8</i>
<i>Implementación.....</i>	<i>9</i>
<i>Aplicación.....</i>	<i>9</i>
<i>Ejemplo 1: Cadenas de Caracteres</i>	<i>9</i>
<i>Ejemplo 2: Números Racionales.....</i>	<i>9</i>
EJERCITACIÓN DE ABSTRACCIÓN.....	10
EJERCICIO 1: POLINOMIO	10
EJERCICIO 2: MATRIZ.....	10
EJERCICIO 3: CONJUNTO	11

Conceptos Generales

Algoritmo

El proceso de resolución de un problema con una computadora conduce a la escritura de un programa y a su ejecución en la misma. Aunque el proceso de diseñar programas es un proceso creativo, se pueden considerar una serie de pasos comunes que deben seguir todos los programadores:

- ◆ Análisis de problema
- ◆ Diseño del algoritmo
- ◆ Codificación
- ◆ Compilación y ejecución
- ◆ Verificación
- ◆ Depuración
- ◆ Documentación

Un algoritmo debe ser preciso (indicar el orden de realización en cada paso), definido (si se sigue dos veces, obtiene el mismo resultado cada vez) y finito.

Paradigmas de Programación

1. Programación por procedimientos
2. Programación modular
3. Abstracción de datos
4. Programación orientada a objetos

Estas son técnicas para programar, paradigmas para escribir “buenos” programas para un conjunto de problemas. Se dice que un lenguaje apoya un estilo de programación si proporciona los recursos que hacen conveniente utilizar ese estilo. El apoyo para un paradigma no viene solo en la forma obvia de los recursos del lenguaje que permiten emplear directamente el paradigma, sino también en la forma más sutil de verificaciones en los tiempos de compilación o ejecución, o ambos, contra desviaciones no intencionales respecto al paradigma.

1. “Decidir qué procedimientos se desean; utilizar los mejores algoritmos que se pueda encontrar”. El programador se concentra en los argumentos de las funciones.
2. “Decidir qué módulos se desean; dividir el programa de modo que los datos queden ocultos en módulos”. Un módulo es un conjunto de procedimientos y funciones afines junto con los datos que manipulan.
3. “Decidir qué tipos se desean; proporciona un conjunto completo de operaciones para cada tipo”. Son tipos definidos por el usuario que se comportan (casi) de la misma manera que los tipos integrados.
4. “Decidir qué clases se desean; proporciona un conjunto completo de operaciones para cada clase; indicar explícitamente lo que tienen en común empleando la herencia”. Cuando existen varios tipos que poseen cosas en común se utiliza el mecanismo de herencia.

Principios de Programación

- § Nombres. Utilice nombres para variables y funciones que intenten explicar su significado y tarea.
- § Documentación. Mantener una documentación precisa pero descriptiva. Esta podrá contener: nombre de programadores fecha y versión; propósito del programa y métodos utilizados; cambios que el programa realiza; referencias a otros programas. Realizar comentarios dentro del programa/ función.
- § Refinamiento. División en funciones/ procedimientos. Top-Down. Especificaciones. Parámetros
- § Input/output. Variables Locales/ Globales (evitar variables globales cuando sea posible) Pre y Post Condiciones.
- § Testeo: proceso de ejecución de un programa con un conjunto de datos elegidos para encontrar errores.

Datos : la calidad de los datos para el testeo es más importante que su cantidad.

Métodos:

- § Black-Box : se trata al programa como una caja negra. El conjunto de datos seleccionado para el testeo deberá contener como mínimo valores fáciles de chequear, valores reales, típicamente utilizados, valores extremos- limites, valores ilegales.
- § Glass-Box: se trata al programa como una caja transparente. El objetivo es elegir un conjunto de datos de tal forma que se cheque cada alternativa que pueda ocurrir.

Tipo de Datos

Es una colección de ítems (elementos) relacionados entre sí, juntos con las operaciones básicas para procesar sus elementos

§ Conjunto de valores

§ Conjuntos de operaciones sobre esos valores

Los valores pueden ser: Atómicos (valores o elementos no divisibles) o Estructurados (valores divisibles y con una relación entre ellos)

Estructura de Datos

Es una colección de datos cuya organización se caracteriza por las funciones de acceso que se usan para almacenar y acceder a los elementos individuales. No se especifica cómo se hace el acceso al elemento, sino que las operaciones devuelven el elemento deseado.

Abstracción

La abstracción, una de las herramientas que más nos ayuda a la hora de solucionar un problema, es un mecanismo fundamental para la comprensión de problemas y fenómenos que poseen una gran cantidad de detalles, su idea principal consiste en manejar un problema, fenómeno, objeto, tema o idea como un concepto general, sin considerar la gran cantidad de detalles que estos puedan tener. El proceso de abstracción presenta dos aspectos complementarios.

1. Destacar los aspectos relevantes del objeto.
2. Ignorar los aspectos irrelevantes del mismo (la irrelevancia depende del nivel de abstracción, ya que si se pasa a niveles más concretos, es posible que ciertos aspectos pasen a ser relevantes).

De modo general podemos decir que la abstracción permite establecer un nivel jerárquico en el estudio de los fenómenos, el cual se establece por niveles sucesivos de detalles. Generalmente, se sigue un sentido descendente de detalles, desde los niveles más generales a los niveles más concretos.

Por ejemplo: los lenguajes de programación de alto nivel permiten al programador abstraerse del sin fin de detalles de los lenguajes ensambladores. Otro ejemplo, la memoria de la computadora es una estructura unidimensional formada por celdas y sin embargo trabajamos como si fuera única. La abstracción nos brinda la posibilidad de ir definiendo una serie de refinamientos sucesivos a nuestro TDA y entiéndase bien que cuando decimos refinamientos sucesivos nos estamos refiriendo a la estrategia que se utiliza para descomponer un problema en sub-problemas. Conforme evoluciona el diseño de software a cada nivel de módulos se representa un refinamiento en el nivel de abstracción. Esto es, incluir detalles que fueron obviados en un nivel superior, en un nivel más bajo de la jerarquía.

Veamos los diferentes tipos de abstracción que podemos encontrar en un programa:

1. **Abstracción funcional:** crear procedimientos y funciones e invocarlos mediante un nombre donde se destaca qué hace la función y se ignora cómo lo hace. El usuario sólo necesita conocer la especificación de la abstracción (el qué) y puede ignorar el resto de los detalles (el cómo).
2. **Abstracción de datos:**
 - § **Tipo de datos:** proporcionado por los lenguajes de alto nivel. La representación usada es invisible al programador, al cual solo se le permite ver las operaciones predefinidas para cada tipo.
 - § **Tipos definidos:** por el programador que posibilitan la definición de valores de datos más cercanos al problema que se pretende resolver.
 - § **TDA:** para la definición y representación de tipos de datos (valores + operaciones), junto con sus propiedades.
 - § **Objetos:** Son TDA a los que se añade propiedades de reutilización y de compartimentación de código.

Si profundizamos más al mundo de la programación y sus conceptos, existen dos de estos conceptos que no se deben confundir, ellos son: tipo de datos y estructura de datos.

Un tipo de dato, en un lenguaje de programación, define un conjunto de valores que una determinada variable puede tomar, así como las operaciones básicas sobre dicho conjunto. Ahora veamos como se van relacionando estos conceptos. Los tipos de datos constituyen un primer nivel de abstracción, ya que no se tiene en cuenta cómo se implementan o se representan realmente la información sobre la memoria de la máquina. Para el usuario, el proceso de implementación o representación es invisible.

Veamos entonces que son las estructuras de datos. Las estructuras de datos son colecciones de variables, no necesariamente del mismo tipo, relacionadas entre si de alguna forma. Las estructuras de datos están caracterizadas por el tipo de dato de los elementos guardados en la estructura y por la relación definida sobre estos elementos.

Al nivel de las estructuras de datos son totalmente irrelevantes las operaciones sobre un elemento en particular, solamente tienen carácter relevante las operaciones que envuelvan la estructura de forma global.

Tipo de Dato Abstracto

En el mundo de la programación existen diversos lenguajes que se han ido creando con el paso del tiempo y que se han perfeccionado debido a las necesidades de los programadores de la época a la que pertenecen. Los primeros lenguajes de programación eran de tipo lineales, ya que un programa se recorría desde un punto marcado como Inicio hasta llegar a un punto Fin. Con el tiempo se fueron creando nuevos lenguajes y en nuestros días los más utilizados son los llamados “Orientados a Objetos”.

Los Lenguajes Orientados a Objetos (LOO) tienen la característica de que no son lenguajes lineales, sino que se forman de diversas funciones, las cuales son llamadas en el orden en que el programa mismo las pide o el usuario determina. Para entender mejor cómo funcionan los Lenguajes Orientados a Objetos, vamos a introducir un concepto fundamental en las Estructuras de Datos denominado Abstracción de Datos y que es parte importante de estos Lenguajes y de la manera en que funciona la mayoría del software comercial de nuestros días.

Historia

El concepto de Tipo de Dato Abstracto (TDA, Abstract Data Types), fue propuesto por primera vez hacia 1974 por John Guttag y otros, pero no fue hasta 1975 que por primera vez Liskov lo propuso para el lenguaje CLU.

Definición

Con mucha frecuencia se utilizan los términos “TDA” y “Abstracción de Datos” de manera equivalente, y esto es debido a la similitud e interdependencia de ambos. Sin embargo, es importante definir por separado los dos conceptos.

La abstracción de datos consiste en ocultar las características de un objeto y obviarlas, de manera que solamente utilizamos el nombre del objeto en nuestro programa. Esto es similar a una situación de la vida cotidiana. Cuando yo digo la palabra “perro”, usted no necesita que yo le diga lo que hace el perro. Usted ya sabe la forma que tiene un perro y también sabe que los perros ladran. De manera que yo abstraigo todas las características de todos los perros en un solo término, al cual llamo “perro”. A esto se le llama ‘Abstracción’ y es un concepto muy útil en la programación, ya que un usuario no necesita mencionar todas las características y funciones de un objeto cada vez que éste se utiliza, sino que son declaradas por separado en el programa y simplemente se utiliza el término abstracto (“perro”) para mencionarlo.

En el ejemplo anterior, “perro” es un Tipo de Dato Abstracto y todo el proceso de definirlo, implementarlo y mencionarlo es a lo que llamamos Abstracción de Datos.

Vamos a poner un ejemplo real de la programación. Supongamos que un pequeño programa saca el área de un rectángulo de las dimensiones que un usuario decida. Pensemos también que el usuario probablemente quiera saber el área de varios rectángulos. Sería muy tedioso para el programador definir la multiplicación de ‘base’ por ‘altura’ varias veces en el programa, además que limitaría al

usuario a sacar un número determinado de áreas. Por ello, el programador puede crear una función denominada 'Área', la cual va a ser llamada el número de veces que sean necesitadas por el usuario y así el programador se evita mucho trabajo, el programa resulta más rápido, más eficiente y de menor longitud. Para lograr esto, se crea la función Área de una manera separada de la interfaz gráfica presentada al usuario y se estipula ahí la operación a realizar, devolviendo el valor de la multiplicación. En el método principal solamente se llama a la función Área y el programa hace el resto.

Al hecho de guardar todas las características y habilidades de un objeto por separado se le llama Encapsulamiento y es también un concepto importante para entender la estructuración de datos.

Caracterización

Un TDA está caracterizado por un conjunto de operaciones (funciones) al cual le denominaron usualmente como su *interfaz pública* y representan el comportamiento del TDA; mientras que la *implementación* como la parte privada del TDA está oculta al programa cliente que lo usa. Todos los lenguajes de alto nivel tienen predefinidos TDA; que son los tipos denominados simples y las estructuras predefinidas, y estos tienen sus interfaces públicas que incluyen las operaciones como la +, -, *, etc.

En un TDA no se necesita conocer como actúan tales operadores sobre la representación interna de los tipos definidos, que además, suele ser una implementación bastante dependiente de la máquina sobre la que trabaje el compilador. Lo interesante es que los lenguajes actuales nos van a permitir ampliar los TDA predefinidos con otros que serán definidos por el propio programador para adecuar así los tipos de datos a las necesidades de los programas.

Los TDA que nos van a interesar de ahora en adelante son aquellos que reflejen cierto comportamiento organizando cierta variedad de datos estructuradamente. A esta forma estructurada de almacenar los datos será a la que nos refiramos para caracterizar cada TDA.

Los TDA que tienen informaciones simples pero dependientes de un comportamiento estructural serán llamados polilíticos y aquellos TDA simples, como son los tipos predefinidos donde la información no es relacionada mediante ninguna estructura y no admiten más que un valor en cada momento serán denominados TDA monolíticos.

Nótese que cuando hablemos de un TDA no haremos ninguna alusión al tipo de los elementos sino tan sólo a la forma en que están dispuestos estos elementos. Sólo nos interesa la estructura que soporta la información y sus operaciones. Para determinar el comportamiento estructural basta con observar la conducta que seguirán los datos.

Caractericemos entonces los TDA. Un TDA tendrá una parte que será invisible al usuario la cual hay que proteger y que se puede decir que es irrelevante para el uso del usuario y está constituida tanto por la maquinaria algorítmica que implemente la semántica de las operaciones como por los datos que sirvan de enlace entre los elementos del TDA, es decir, información interna necesaria para la implementación que se esté haciendo para ese comportamiento del TDA. Resumiendo podemos decir, que tanto la implementación de las operaciones como los elementos internos del TDA serán privados al acceso externo y ocultos a cualquier otro nivel.

Un TDA representa una abstracción:

- § Se destacan los detalles (normalmente pocos) de la especificación (el qué).
- § Se ocultan los detalles (casi siempre numerosos) de la implementación (el cómo).

Principales características

Ocultamiento de la información

Todos los detalles de la representación e implementación están ocultos.

Protección de la manipulación

La única manera que el usuario puede manipular los datos o acceder a ellos es a través de las operaciones especificadas. Podemos asegurar que dadas las pre-condiciones se cumplirán las post-condiciones, es decir el uso está libre de errores.

Simplicidad

Separar las cualidades esenciales del dato, su estructura y operaciones de los detalles que no hacen al tipo de dato.

Integridad

Al implementar las estructuras de datos abstractas usando operaciones que actúan como cajas negras, donde el usuario no puede ver su interior, sino sólo a través de las primitivas, esta asegurada la integridad de los datos.

Implementación Independiente

Los módulos son fácilmente modificables, es decir podemos cambiar la representación e implementación sin que las aplicaciones desarrolladas se vean afectadas.

Objetivos

Ocultar detalles de representación e implementación. Encapsular el tipo de dato.

Ventajas

- Ø Especificaciones precisas (pre y post)
- Ø Modularidad
- Ø Simplicidad
- Ø Integridad
- Ø Independencia
- Ø Cambios de implementación independientes

Etapas en la construcción de un TDA

- Ø Elementos componentes del tipo de dato
- Ø Estructura relación entre los elementos
- Ø Dominio valores a tomar
- Ø Operaciones Especificación

Pre-condición es la condición que debe cumplirse para que la operación pueda realizarse.

Post-condición es el estado en que va a quedar la estructura y elementos asociados luego de realizar la operación.

Representación

Es la forma de guardar los valores del tipo abstracto. En esta etapa se define el lenguaje de implementación, y se usan sus tipos básicos (int, char, array, struct, etc).

Implementación

En esta etapa se codifican todas las operaciones abstractas definidas.

Aplicación

Utilización de las operaciones abstractas por parte del usuario del tipo de dato. La comunicación entre el nivel de aplicación y el nivel de implementación debe hacerse sólo mediante las interfaces especificada en los procedimientos y funciones de acceso (primitivas). Esto permite que la implementación podría cambiarse completamente sin que afecte el uso del tipo de dato en las aplicaciones.

Ejemplo 1: Cadenas de Caracteres

Primitivas

1. void Cadena_Crear(cadena *S);
2. void Cadena_Sacar(cadena *S, char *c);
3. void Cadena_Agregar(cadena *S, char c);
4. int Cadena_Longitud(cadena S);
5. int Cadena_Vacia(cadena S);
6. int Cadena_Llena(cadena S);

Tipo de Datos

```
struct cadena
{
    long largo;
    char dato[100];
}
```

Ejemplo 2: Números Racionales

Primitivas

1. void Racional_Crear(int n, int d, racional *R);
2. void Racional_Simplificar (racional *R);
3. int Racional_Mayor(racional R1, racional R2);
4. int Racional_Igual(racional R1, racional R2);
5. long Racional_Denominador(racional *R);
6. long Racional_Numerador(racional *R);

Tipo de Datos

```
struct racional
{
    int num;
    int den;
}
```

Ejercitación de Abstracción

Ejercicio 1: Polinomio

Se desea definir e implementar un tipo de dato abstracto POLINOMIO. Este tipo podría mantener polinomios de grados entre N y 0 (N cte fija). Se definieron algunas primitivas básicas a partir de las cuales se desea desarrollar operaciones básicas entre polinomios. Las primitivas de esta estructura son:

```
void Cargar_Coeficiente(T_Pol *P, int Exp, int Coef)
int Obtener_Coeficiente(T_Pol P, int Exp)
int Obtener_Grado(T_Pol P)
```

Se pide:

§ Completar las primitivas, definiendo nuevas si lo cree necesario, parámetros, pre y post condiciones de cada una.

§ Desarrollar los siguientes procedimientos abstractos:

```
void Suma_Polinomio (T_Pol p1, T_Pol p2, T_Pol *p3)
void Polinomio_Por_Constante(T_Pol *p1 ,float k)
```

Definir una representación para el TDA polinomio y luego implementar todas las primitivas definidas en el punto a

Ejercicio 2: Matriz

Sea un TDA “matriz” de reales, con las siguientes primitivas:

```
void matriz_asignar(matriz *m, int I, int j, float v);
pre: m esta creada con maxfilas>= i y maxcolumnas>=j
pos: la celda de m(i,j) tiene el valor v.
```

```
float matriz_obtener_celda(matriz m, int i, int j);
pre: m esta creada con maxfilas>= i y maxcolumnas>=j
pos: devuelve el contenido de la celda de m(i,j).
```

```
void matriz_dimensiones(matriz M, int *maxfilas, int
*maxcolumnas);
pre: m esta creada.
pos: devuelve los valores maxfilas y maxcolumnas con que fue
creada la matriz.
```

Se pide:

§ Desarrollar los siguientes procedimientos abstractos:

```
void producto_matricial(matriz m1, matriz m2, matriz *m3, int *e);
```

pre: m1 y m2 están creadas

post: m3 tiene el producto de m1 x m2. Si este no es posible, devuelve error.

```
float suma_diagonal_principal(matriz m);
```

pre: m esta creada y debe ser una matriz cuadrada

post: devuelve la suma de los elementos de la diagonal principal

```
void máximo(matriz m, int *i, int *j, float *v);
```

pre: m esta creada

post: devuelve en v el máximo valor de la matriz y las coordenadas i y j de dicho valor.

```
void suma_de_matrices( matriz m1,matriz m2, matriz * m3, int *e);
```

pre: m1 y m2 están creadas

post: m3 tiene el resultado de m1 + m2. si este no es posible, devuelve error.

```
void matriz_transpuesta(matriz m1,matriz *m2);
```

pre: m1 esta creada

post: m2 tiene la matriz transpuesta de m1.

§ Sugerir un posible tipo de datos para la implementación del TDA matriz, e implementar la primitiva `matriz_crear` y `matriz_asignar`.

Ejercicio 3: Conjunto

Sea un TDA “t_conj” que almacena elementos de tipo ‘tipo_elem’, que tiene una función:

`int elem_clave( tipo_elem E)` que devuelve la clave del elemento, y con las siguientes primitivas:

1. `void conjunto_crear (t_conj *c);`
2. `void conjunto_agregar (t_conj *c, tipo_elem e);`
3. `int existe_elemento (t_conj c, tipo_elem e);`
4. `int conjunto_vacio (t_conj c);`
5. `void conjunto_borrar(t_conj *c,tipo_elem e);`
6. `void conjunto_interseccion(t_conj c1, t_conj c2, t_conj *c3);`
7. `void conjunto_union(t_conj c1, t_conj c2, t_conj *c3);`
8. `void conjunto_diferencia(t_conj c1, t_conj c2, t_conj *c3);`

9. `int conjunto_lleno(t_conj c);`

Se pide:

- § Especificar pre y post condiciones de las primitivas utilizadas en los puntos anteriores.
- § Implementar las primitivas.